Chapter One

Data Structure and Algorithms

Prepared by Bilew A.

Course Outline

- Overview of DSA
- Algorithm Analysis
- Introduction – Mathematical function
- General Big-Oh rules
- Language Features to Support
- Comparison of Algorithms(Big-O Notation)

Overview of DSA

- **Data Structures and Algorithms (DSA)** teaches you how to think and solve complex problems systematically.
- Using the right data structure and algorithm makes your program run faster, especially when working with lots of data.
- Knowing DSA can help you perform better in **job interviews** and land great jobs in **tech companies.**

Overview of DSA

- DSA is fundamental in software world.
  - ✓ OS
  - ✓ Database System
  - ✓ We App
  - ✓ Machine Learning
  - ✓ Video Games
  - ✓ Cryptographic system
  - ✓ Data Analysis
  - ✓ Search engine

Overview of Data Structure

- To understand data structures, we must first define how data is organized at a fundamental level:
  - ✓Data
  - ✓Attribute & Entity
  - ✓Record
  - ✓File

Overview of Data Structure

❖ **What is Data ?**

- ✓ Data can be defined as the elementary value/collection of values.
- ✓ **example:-** Student's name and its id

What is Record ?

- ✓ Record can be defined as collection of various data items

Overview of Data Structure

- **Example:** student entity, name, address, course and marks can be grouped together to form record.
- **What is File?**
  - ✓ **file** is a collection of various records of one type of entity.
  - ✓ **Example:** if there are so employees in class, then there will be 20 records in related file where record contains information of employee.

Overview of Data Structure

❖ **What is Attribute and Entity?**

- ✓ An entity represents class of certain objects.
- ✓ It contains various attributes each attribute represents particular property of that entity.

What is a Data Structure?

- Data Structure is a systematic way to store, organize, and represent data in a computer's memory (or disk storage) so that it can be used efficiently.
  - ✓ The data structure is not any programming language like C, C++, Java etc.
  - ✓ It is set of algorithms that we can use in any programming language to structure data in memory.
- The Need for Data Structures: The primary purpose of most computer programs is not just to perform calculations, but to store and retrieve information as fast and efficiently as possible.

Classification of Data Structures

- **Linear Data Structures:** Data is arranged sequentially. Each element connects to exactly one adjacent element.
  - ✓ It also classified as Static DS (Array) and Dynamic DS (Queue, Stack, Linked list)
- **Non-Linear Data Structures:** Elements are arranged randomly or hierarchically, where one element can connect to 'n' number of elements.
  - ✓ It also classified as tree and Graph

Algorithm Analysis

- An **algorithm** is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer.
- It is not complete program or code; it is just a solution (logic) of a problem, which can be represented using:
  - ✓ a flowchart or pseudocode.

Algorithm Analysis

❖ **The major categories of algorithms are given below:**

- ✓ **Sort:-** Algorithm developed for sorting the items in a certain order.
- ✓ **search:-** Algorithm developed for searching the items inside a data Structure.
- ✓ **Delete:-** Algorithm developed for deleting the existing element from the data structure.
- ✓ **Insert:-** Algorithm developed far inserting an Item inside a data structure.
- ✓ **Update: -** Algorithm developed for updating the existing element inside a data structure.

Algorithm Analysis

- **Algorithm analysis** is the process of **evaluating the efficiency** of an algorithm in terms of **time** and **space complexity**.
- The aim is to understand how an algorithm behaves as the input size increases, helping to choose the most efficient solution for a problem.
- The algorithm can be analyzed in two levels i.e.
  - ✓first is before creating the algorithm
  - ✓Second is after creating the algorithm.

Algorithm Analysis

There are two analysis of an algorithm.

- ✓**Priori Analysis :-** It is the theoretical analysis of an algorithm which is done before implementing the algorithm.
- ✓**Posterior Analysis :-** It is a practical analysis of an algorithm.
- ✓The practical analysis is achieved by implementing algorithm using any programming language.

Algorithm complexity

❖ **The performance of the algorithm can be measured in two factors:**

1. Time complexity

- ✓ The time complexity of an algorithm is the amount of time required to complete the execution.
- ✓ The time complexity is denoted by the big o notation of an algorithm.
- ✓ Big o notation is the asymptotic notation to represent time complexity.
- ✓ The time complexity is mainly calculated by counting the number of steps to finish execution.

Algorithm complexity

2. Space complexity

- ✓ An algorithm's space complexity is the amount of space required to solve a problem and produce an output.
- ✓ Similar to the time complexity, space complexity is also expressed in big o notation.
- ✓ **Space complexity = Auxiliary space + Input size.**

Algorithm complexity

The time required by an algorithm comes under three types:

- ✓**Worst case :-** It defines the input for which the algorithm takes a huge time.
- ✓**Average case :-** It Takes average time for the program execution.
- ✓**Best case:-** It defines the input for which the algorithm takes the lowest time.

Algorithm complexity

Asymptotic Notations

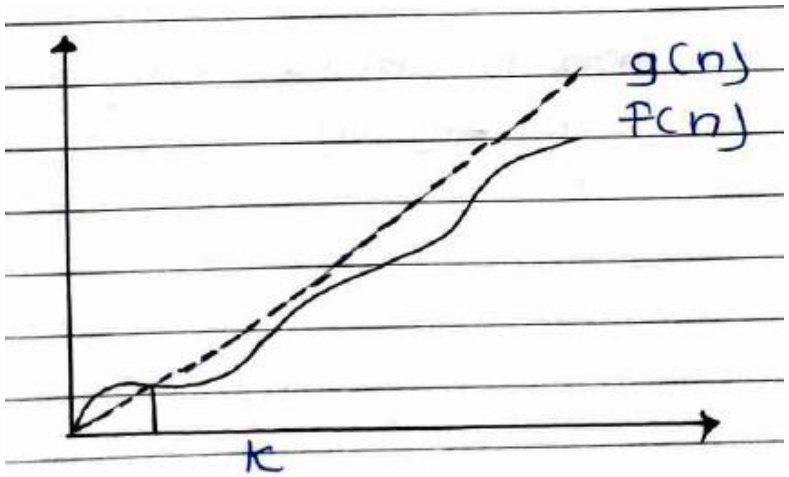
- The commonly used asymptotic notations used for calculating the running time complexity of an algorithm is given below:-

1. Big o notation (O)

✓ This measures the performance of an algorithm by simply providing the order of growth of the function.

✓ This notation provides an **worst case** (upper bound) on a function which ensures that function never grows faster than the upper bound.

Algorithm complexity



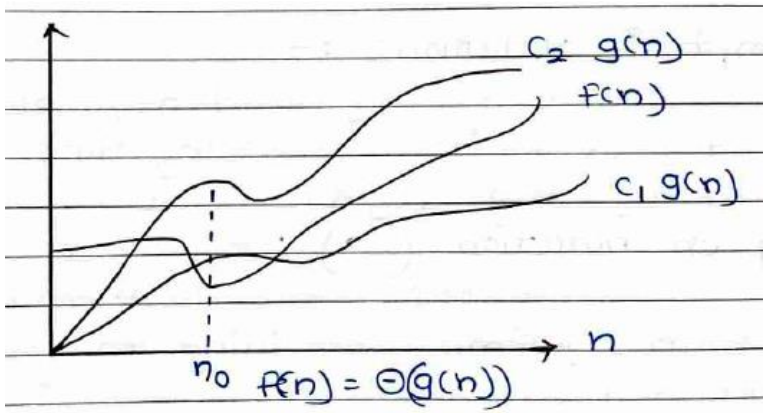
- Example: If f(n) and g(n) are two functions defined for positive integer,
- Then f(n) = o g(n) as f(n) is big oh of g(n) or f(n) is an order of g(n)) if there exists constants c and no such that:
- f(n) <= c.g(n)

Algorithm complexity

2. Omega Notation (Ω):-

- ✓ It basically describes **best case (lower bound)** scenario which is opposite to big o notation.
- ✓ It is the formal way to represent lower bound an algorithm's running time.
- ✓ Example: if f(n) and g(n) are two functions defined for positive integers.
- Then f(n)=Ω g(n) as f(n) is omega of g(n) or f(n) is the order of g(n) if there exists constants c and no such that

  c1. g(n) <= f(n) <=c2.g(n)



Algorithm complexity

3. Theta Notation

- ✓ The theta notation mainly describes **average case** (tight bound) scenarios
- ✓ It represents realistic time, complexity of an algorithm.
- ✓ Big theta is mainly used when the value of worst case and best case is same.

Introduction – Mathematical function

- **Mathematical functions** play a **crucial role** in **analyzing algorithm performance.**
- They help describe the **running time** of an algorithm in terms of the **input size**, usually denoted as n.
- **Some common mathematical functions used in algorithm analysis include:**

1. Constant Function:

- ✓ A function that always takes the same amount of time, regardless of input size.
- ✓ Example: Accessing an element in an array by index.
- ✓ $f(n) = c$, Where C is constant

Introduction – Mathematical function

2. Linear Function: $O(n)$

- ✓A function that grows proportionally with n
- ✓**Example:** Looping through an array of size n

f(n)=an + b, Where a and b are constants.

3. Quadratic Function: $O(n^2)$

- ✓A function where the running time grows with the square of n.
- ✓**Example:** Nested loops iterating over an array.

f(n)=a n^2 +bn+c, Where a, b, and c are constants

Introduction – Mathematical function

4. Logarithmic Function: O(log n)

- ✓A function that grows slowly as n increases.
- ✓**Example:** Binary search, where the input size is halved in each step.

$$f(n)= a \log n + b$$

5. Exponential Function: O(2n)

- ✓A function that doubles for each increase in n.
- ✓**Example:** Recursive solutions to the Fibonacci sequence.

$$f(n)= \mathbf{2^n}n+b$$

- ✓These functions helps in evaluating how an algorithm scales with input size.

Introduction – Mathematical function

The General Big-Oh Rules include:

1. Drop Constants

- ✓If f(n)=5n+3, the Big-O notation is O(n), ignoring constant factors.

2. Drop Non-Dominant Terms

- ✓If f(n)= n^2+3n+5, the dominant term is n^2, so O(n^2)

3. Multiplication rule

- ✓If an algorithm has two independent steps, their complexities multiply.
- ✓**Example:** A loop of size n inside another loop of size m-> O(n . m).

Introduction – Mathematical function

❖Addition rule

- If an algorithm has sequential steps, their complexities are added.
- Example: one loop runs in O(n), another in o(n2), so total = O(n+n2)
- Generally, understanding **mathematical functions** helps in predicting algorithm performance,
- While **Big-O rules** allow us to simplify and compare complexities. These concepts are essential for designing efficient algorithms.

Basic Terminologies

Term	Description
Algorithm	A set of step-by-step instructions to solve a specific problem.
Data Structure	A way of organizing data so it can be used efficiently. Common data structures include arrays, linked lists, and binary trees.
Time Complexity	A measure of the amount of time an algorithm takes to run, depending on the amount of data the algorithm is working on.
Space Complexity	A measure of the amount of memory an algorithm uses, depending on the amount of data the algorithm is working on.
Big O Notation	A mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity.

Reading Assignment

1. Language Features to Support (Implementation of DSA)
2. Comparison of Algorithms(Big-O Notation)
3. What are the best IDEs for Data Structures and Algorithms?